Unsupervised Learning

Machine Learning

Chicago Booth, Chicago, IL, USA, Spring 2025

Christian Hansen
The University of Chicago
Booth School of Business

Damian Kozbur
University of Zurich
Department of Economics

Introduction to unsupervised learning

- **Unsupervised learning** is a data analysis method which (semi)automatically delivers useful descriptions of data
  - Useful for data exploration and visualization
  - Can be used to inform or combine with supervised learning

- Examples of Unsupervised Learning
  - Density estimation
  - Clustering
  - Dimension reduction

Lecture outline

Part I: **Clustering**

- Overview
- The *k*-means method
- Application 1: Segmenting Illinois zip codes

Part II: **Encoding**

- Dimension reduction
- Application 1 continued: Segmenting Illinois Zipcodes
- Other dimension reduction techniques

Part III: **Recommender Systems**

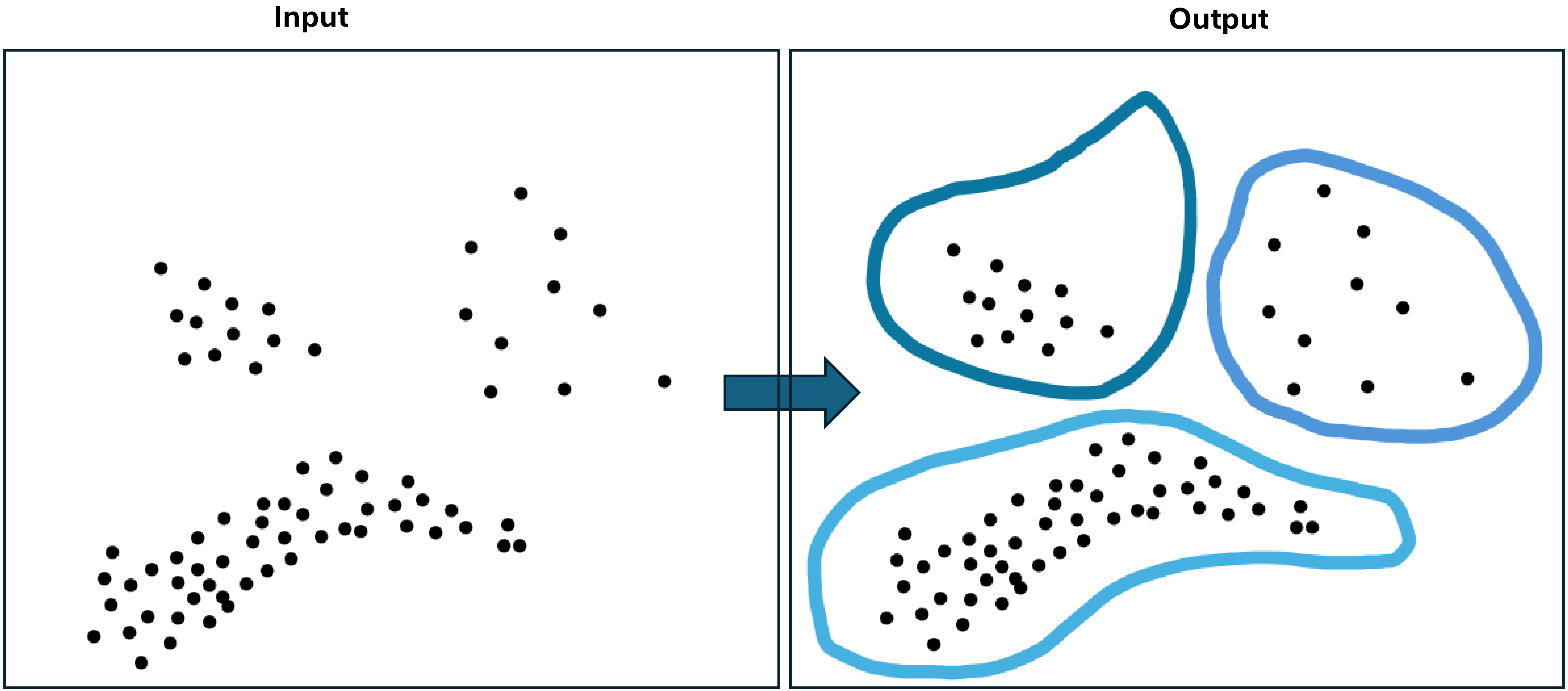
- Application 2: Recommender systems for Swiss wines via clustering
- Application 3: Recommender systems for team building via principal components with supervised learning

Clustering

Part I – Clustering: Purpose, Methods, Applications

Clustering

- Clustering is any method for automatically partioning data



Predicting versus Assigning *Y*

- In supervised learning, there is data about *X* and *Y*
  - Supervised learning delivers a rule *f* which predicts *Y* from *X*

$$Y = f(X) + \varepsilon$$

$$f \text{ minimizes } L(g, \text{Data}) \text{ over possible } g$$

  - E.g., for quadratic loss, $L(g, \text{Data}) = \sum_{\text{Oservations } i \text{ in Data}} (\, Y_i - g(\, X_i \,)\,)^2$

- Unspervised learning is similar, but there is no *Y* variable in data
  - Produce an assignment to a new auxiliary *Y*-like variable

$$f(X) = \{\ ■,\ ■,\ ■\ \}$$

General Description of Clustering

- **Goal**
  - Automatically and reliably group “alike” data observations together into clusters

- **Needed to Operationalize**
  - A numerical definition of “likeness” called *d* (short for “distance”)
  - A clustering Algorithm that works with *d*

- **Applications Overview**
  - Recommendor systems
  - Market Segmentation
  - Scientific classification
  - In general, different analytics can now be applied to different clusters

k-means for 2-D data

***k*-means** is a commonly used clustering method defined using

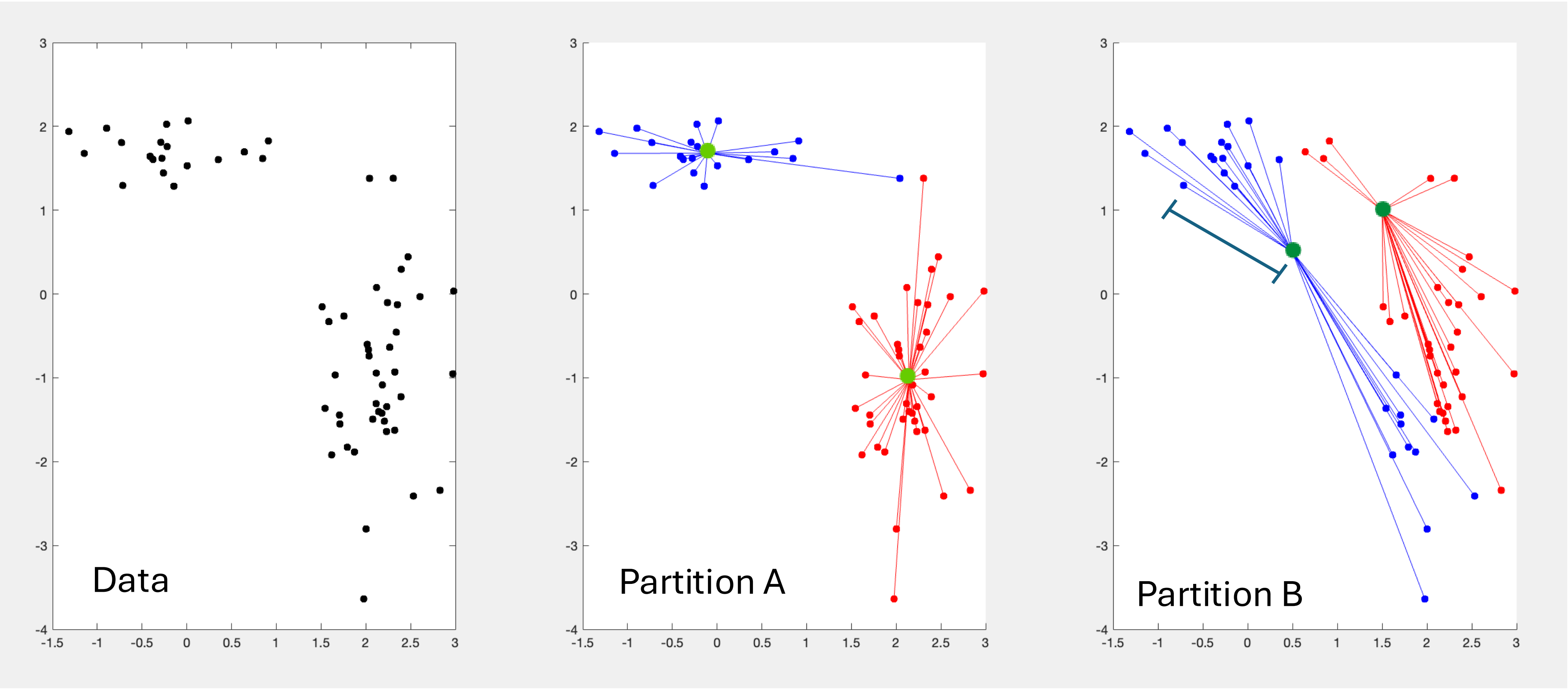
- **Distance** *d*: For 2-D data, *d* can be standard Euclidean distance
  - For observations *i* and *j*, the distance *d*(*i*, *j*) is defined by

$$d(i, j) = \sqrt{(\text{x-coord}(i) - \text{x-coord}(j))^2 + (\text{y-coord}(i) - \text{y-coord}(j))^2}$$

- **Clustering Algorithm:** Minimize **Obejctive Function** *L* :

$$L(\text{Data, Cluster centers}) = \sum_{\text{Oservations } i \text{ in Data}} d(i, i\text{'s nearest center})$$

Visualization of the math for two partitions



-candidate cluster centers for Partition A

-candidate cluster centers For Partition B

Partition A: Sum of distances to cluster centers = 12.5419 + 40.5925 = 53.1344

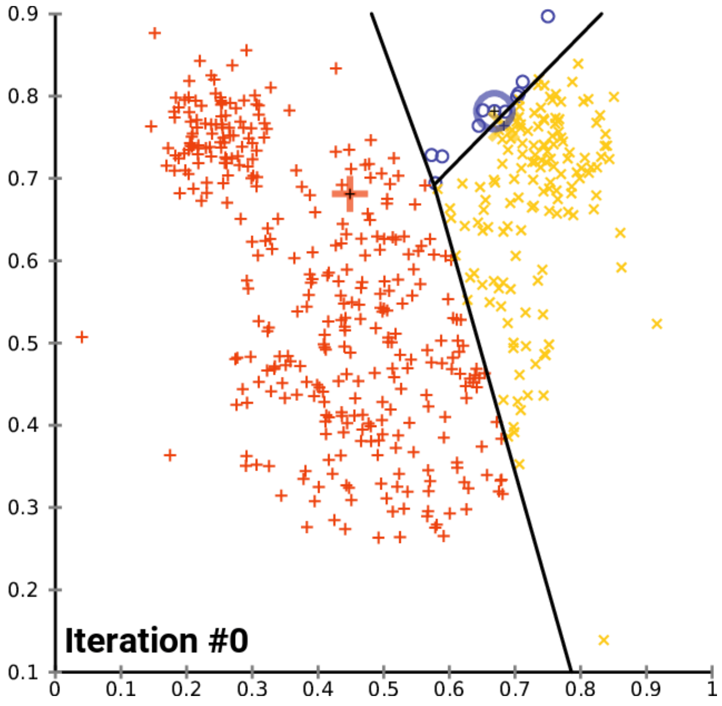
Partition B: Sum of distances to cluster centers = 50.1245 + 57.3321 = 107.4566

Task: Choose the best cluster centers.
According to sum of distances criteria, which is prefered above, partition A or B?

Conclusion: Of the two choices, minimizing distance sums leads to choice of **Partition A** as winner

Practical compution of *k*-means partition

- **Loyd's algorithm**
  - Start with random initial partition
  - Calculate best center for each current cluster
  - Reassign observations to closest cluster center
  - Repeat until clusters do not change
- GIF demonstrates convergence to a better partition



A few disadvantages of *k*-means

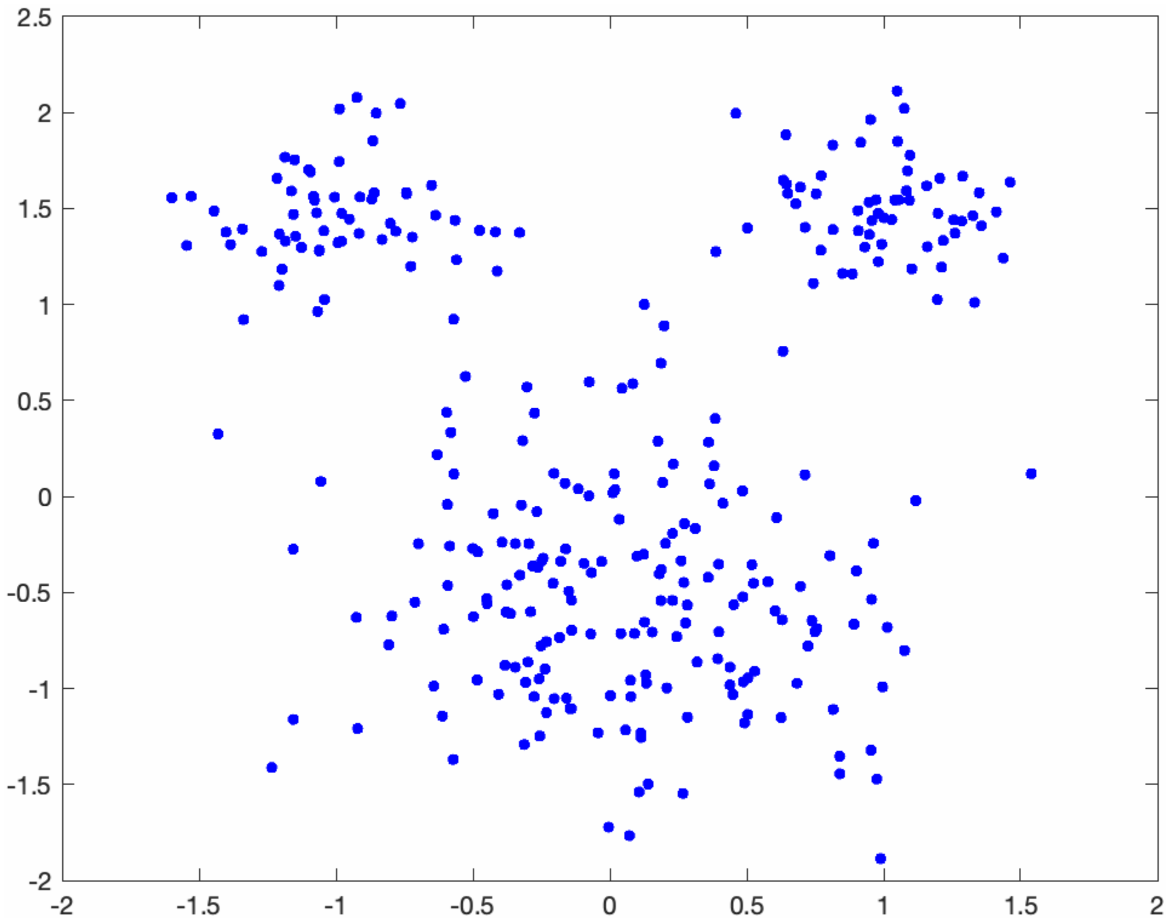
- *k* –means works best if clusters have comprable “within”-variances
- Loyd’s algorithm is generically fast and works on almost all datasets which haven’t been specially engineered, but it can get stuck in loops
- Loyd’s algorithm falls into local minima – which means that running the algorithm several times can lead to different partitions because of initial conditions (where the algorithm starts matters)
- Alternatives to *k*-means include hierarchical (agglomerative) clusters

Hierarchical clustering

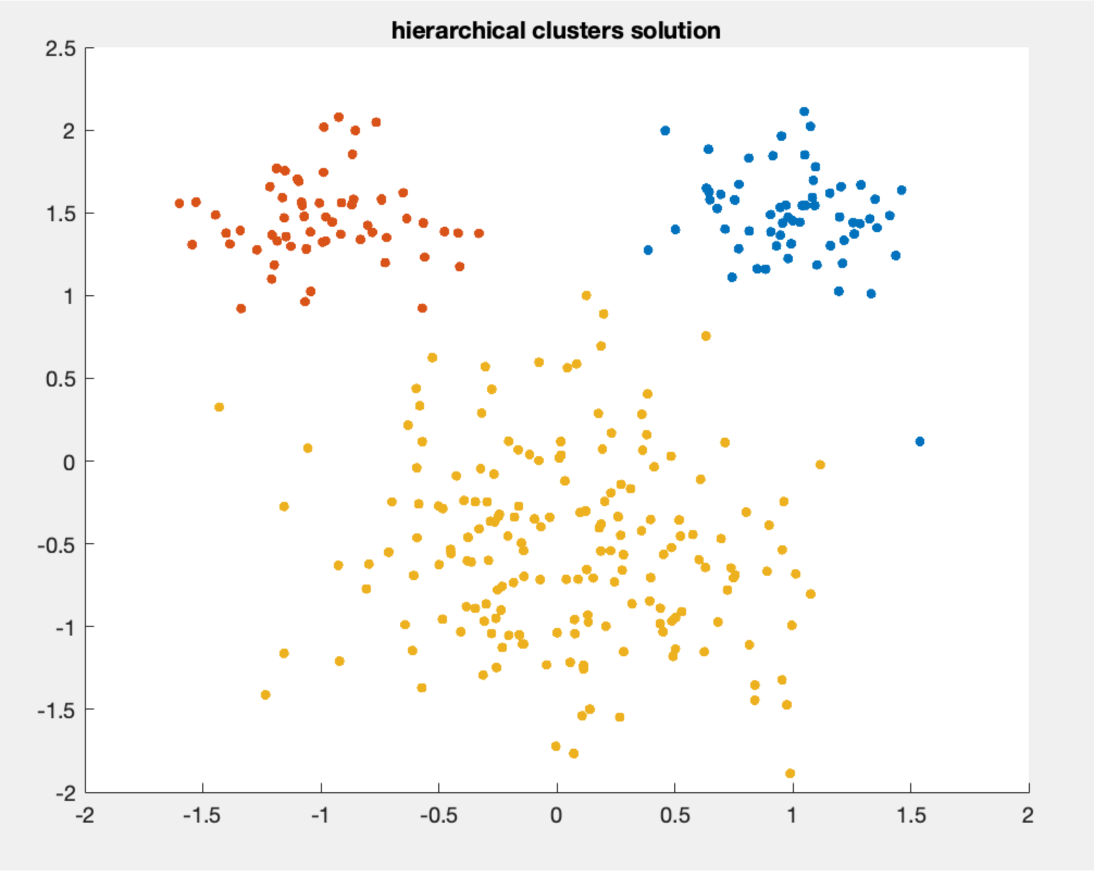
- Builds up clusters “from the bottom up”
- Two closest (there are several possible definitions of closest commonly used) clusters a merged
- Still requires a user supplied notion of distance between observations

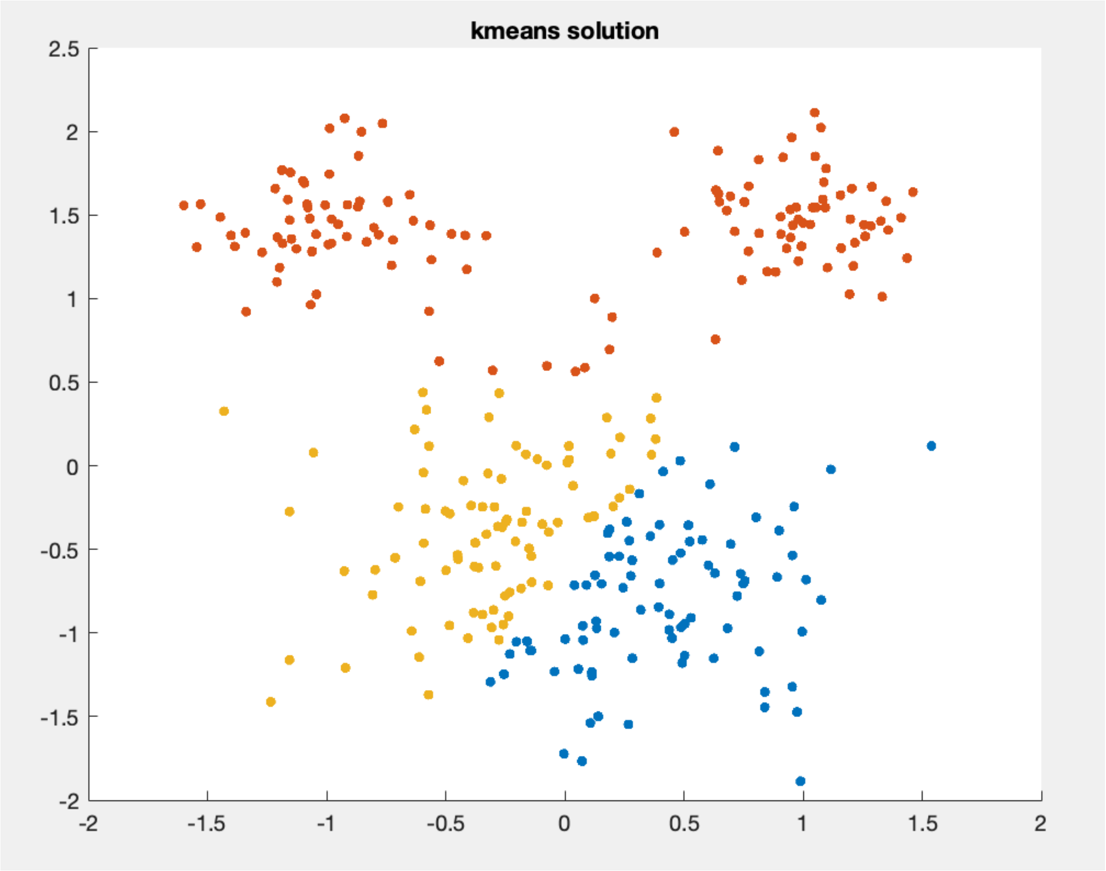
Hierarchical clustering illustration

- **Agglomarative cluster algorithm**
  - Start with every point in its own cluster
  - Merge clusters with smallest resulting diameter
  - Repeat until desired number of clusters appear
- GIF demonstrates path to final partition



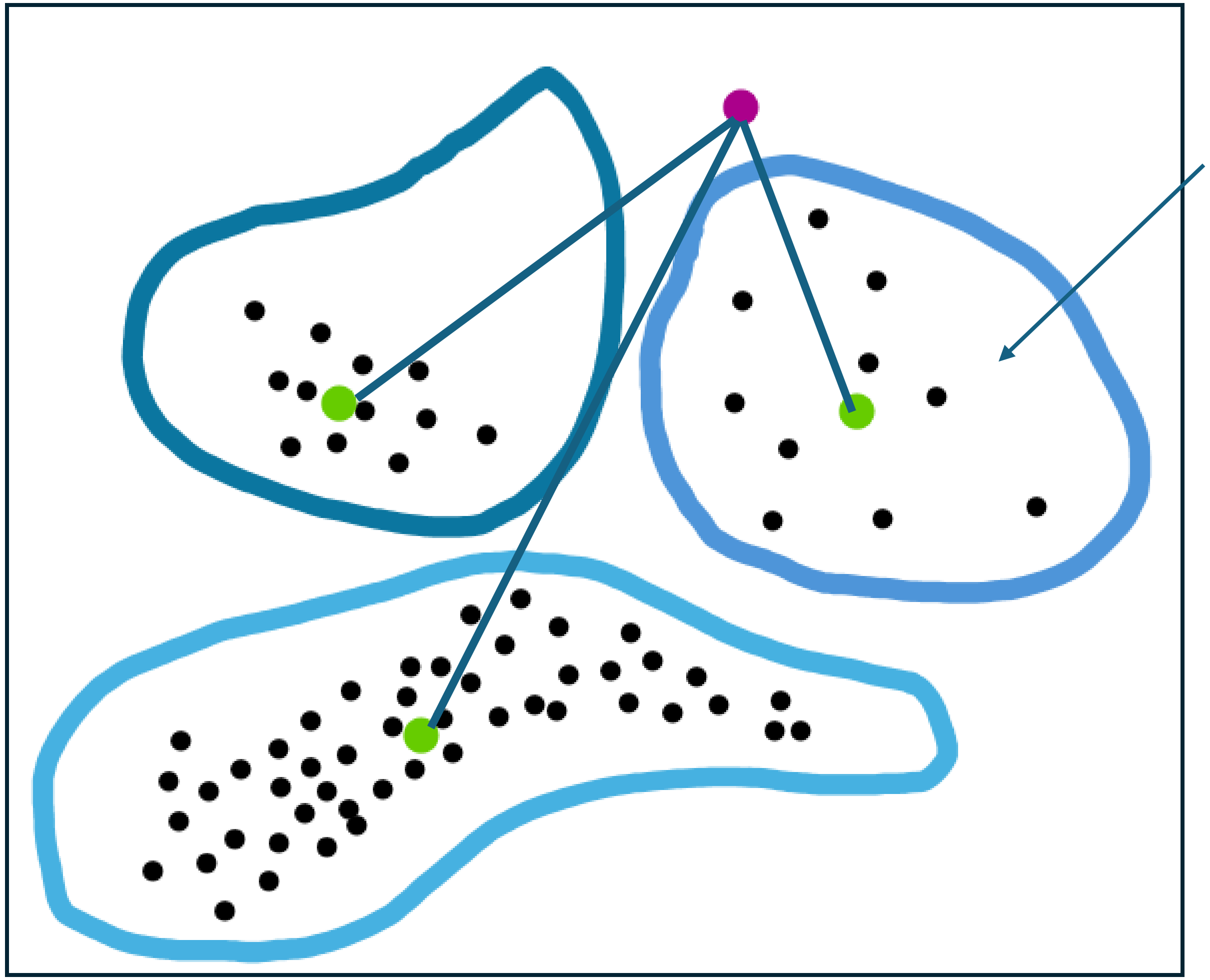
Resulting *k*-means versus hierarchical clusters





Classifying new observations

- Once *k*-means is run once, new incoming observations can be assigned based on proximity to cluster centers without needing to recalculate the entire procedure



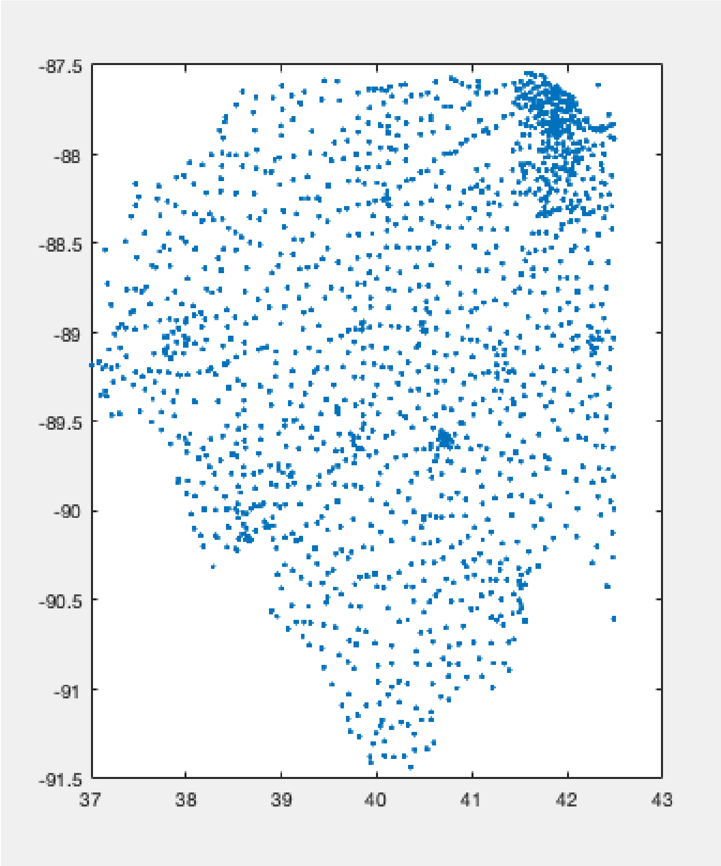
* New purple observation assigned to cluster with nearest cluster center

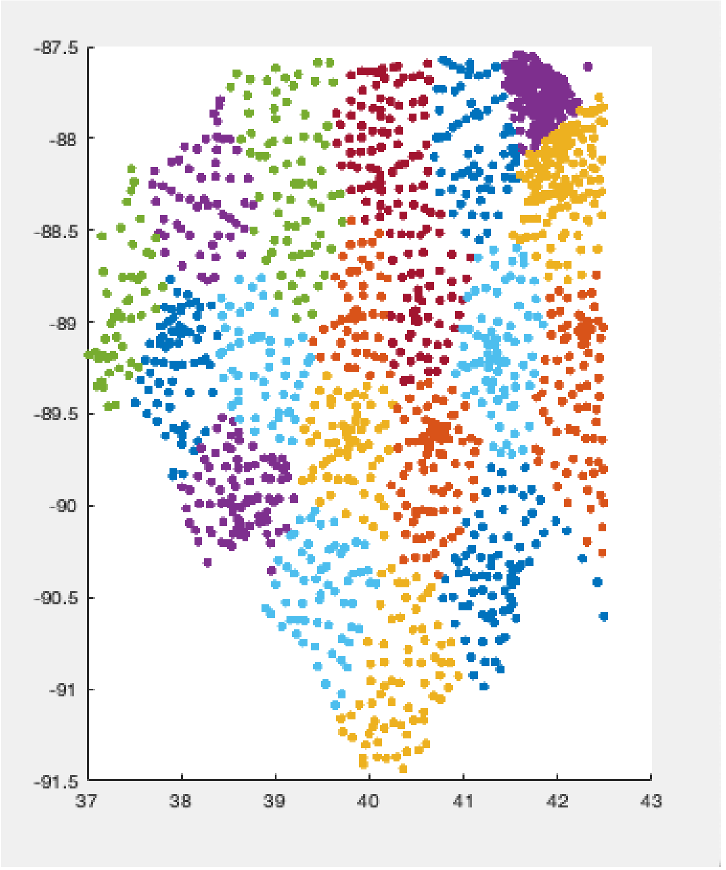
Moving beyond 2-dimensional data:
On the choice and important of distance

- When data are coordinates in a 2-D plane, eyeball method can also be quite reliable (though not automatable)
- When data attribute are not locations in a plane, need a different notion of "alike" with which to calculate
- Choice of distance will depend on ultimate need for clustering and application

Segmenting Illinois Zipcodes

- Cluster Illinois zipcodes using latitude and longitude, *k* = 20





k-means with 20 clusters

Euclidean Distance for data with many numerical attributes

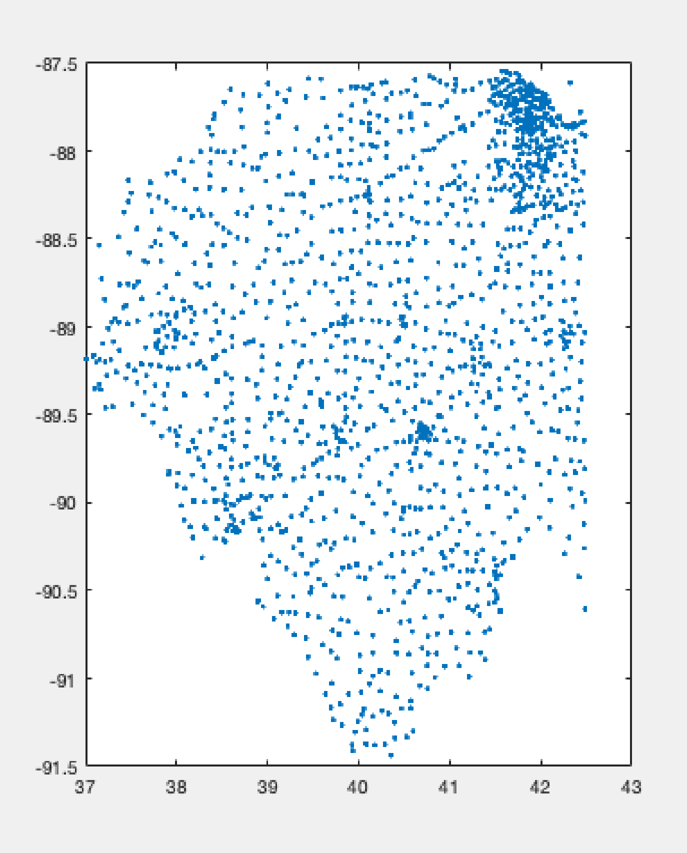
- *k*-means can be used if data observations have a distance *d*
- Suppose *i* and *j* are row labels,
- Numberical attributes (or data columns) are $x_1, x_2, \ldots, x_p$
- Euclidean distance is

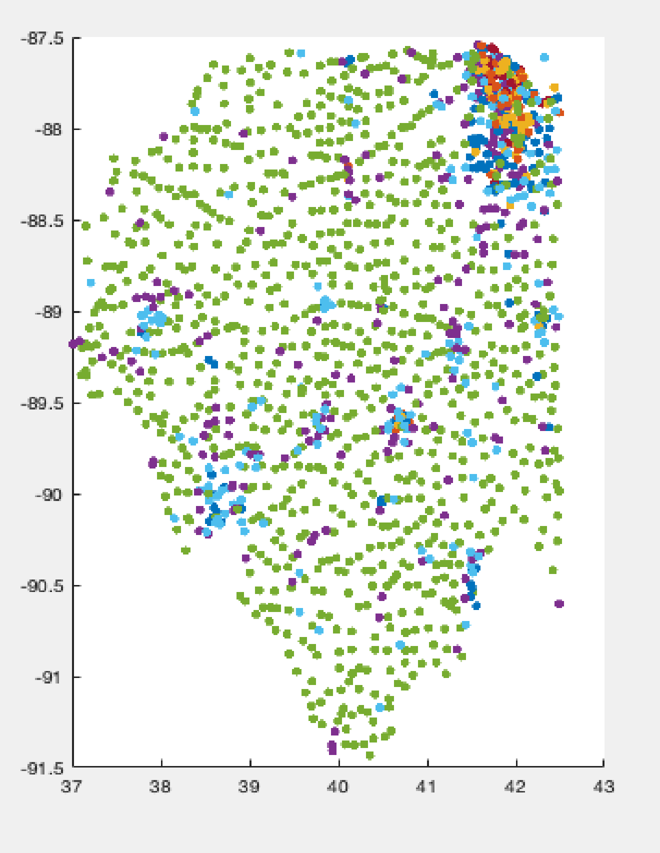
$$d(i, j) = \sqrt{(x_1(i) - x_1(j))^2 + (x_1(i) - x_1(j))^2 + \ldots + (x_p(i) - x_p(j))^2}$$

- Defining a distance is important for data which is not exclusively geographic

On using additional variables for distance

$$d(\text{ Zipcode A, Zipcode B })^2 = (\text{Latitude A} - \text{Latitude B})^2 + (\text{Longitude A} - \text{Longitude B})^2 + (\text{Pop Density A - Pop Density B})^2$$



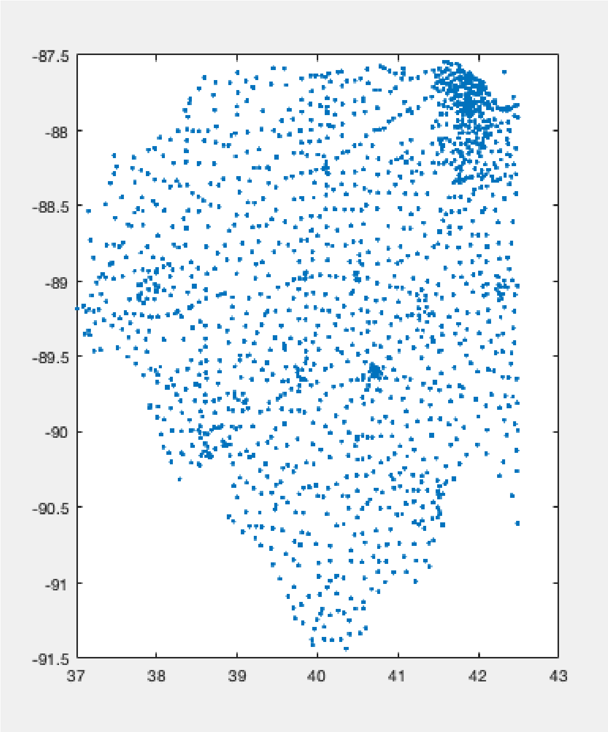


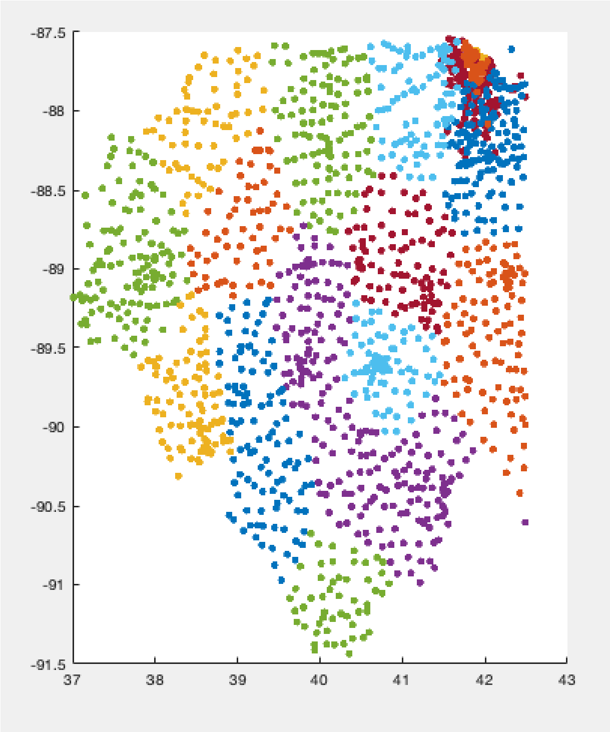
k-means with 20 clusters

On scaling variables in distance choices

- Normalizing population density were by standard deviation

$$d(\text{ Zipcode A, Zipcode B })^2 = (\text{Latitude A} - \text{Latitude B})^2 + (\text{Longitude A} - \text{Longitude B})^2 + (\text{Pop Density A - Pop Density B})^2/\text{stdev}(\text{Pop Density})^2$$

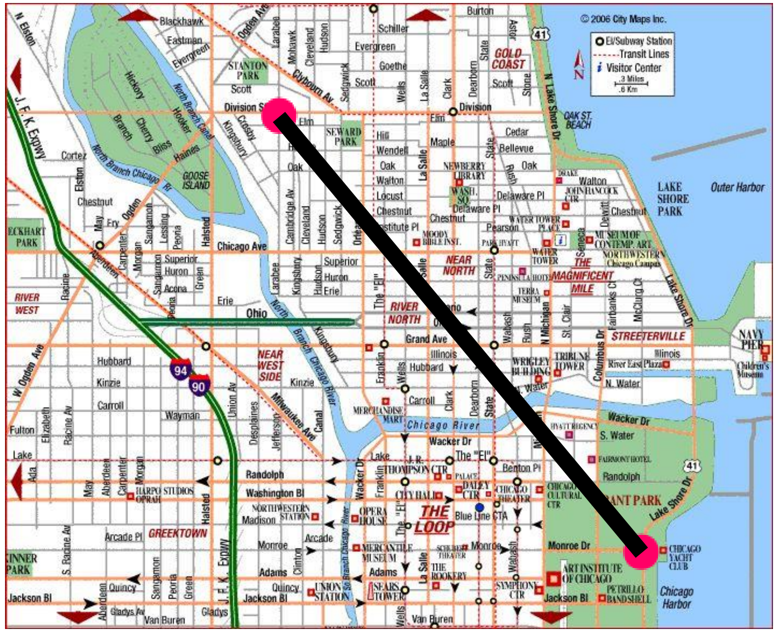


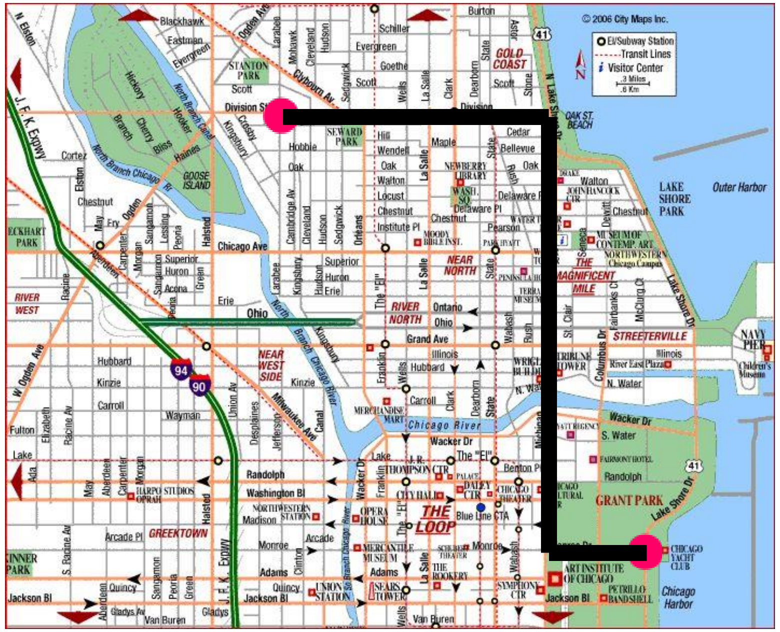


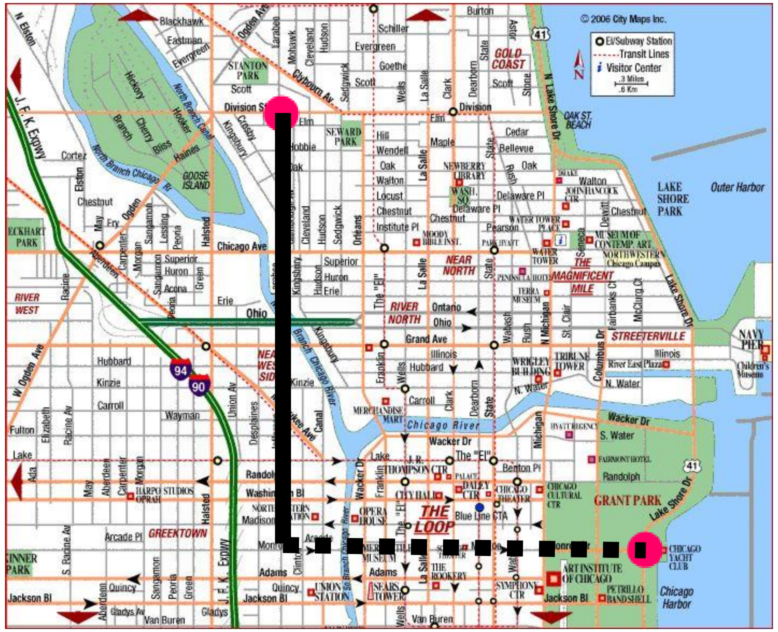
k-means with 20 clusters

Most common distances for coordinate data

- Euclidean Distance
  (aka Crow-flies distance or L2 distance)
- Manhattan Distance
  (aka L1 distance)
- Sup Distance
  (aka L∞ distance)

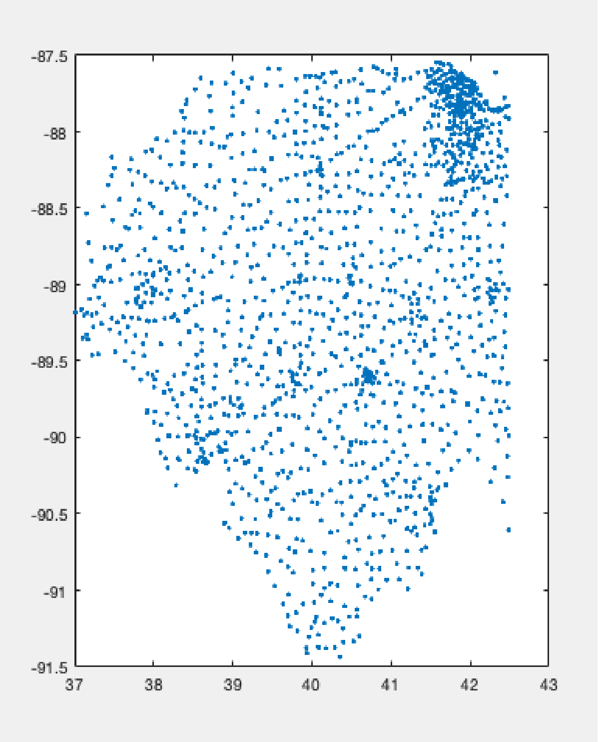


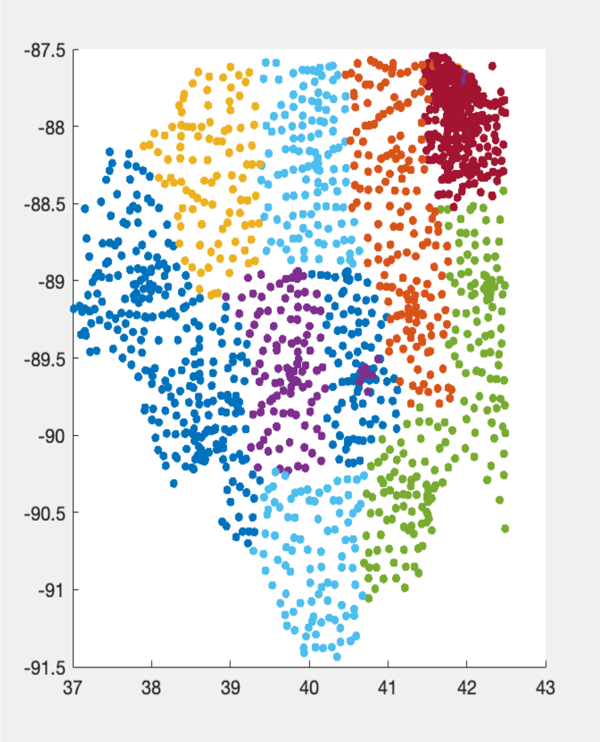




Using Sup distance for Lat/Long andUnivariate Euclidean distance for Population Density

d(Zipcode A, Zipcode B) = max(|Latitude A – Latitude B|, |Longitude A – Longitude B|) + |Pop Density A - Pop Density B| /stdev(Pop Density)



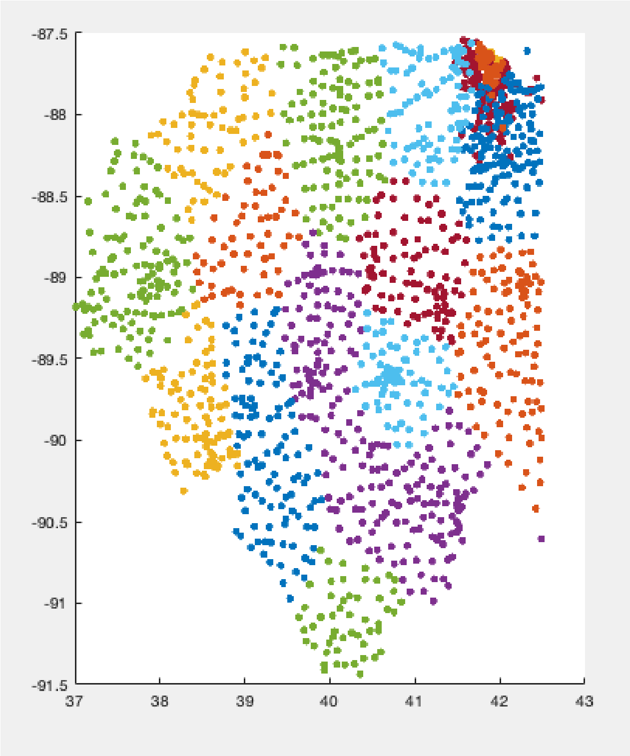


Resulting partition respects north-south boundaries

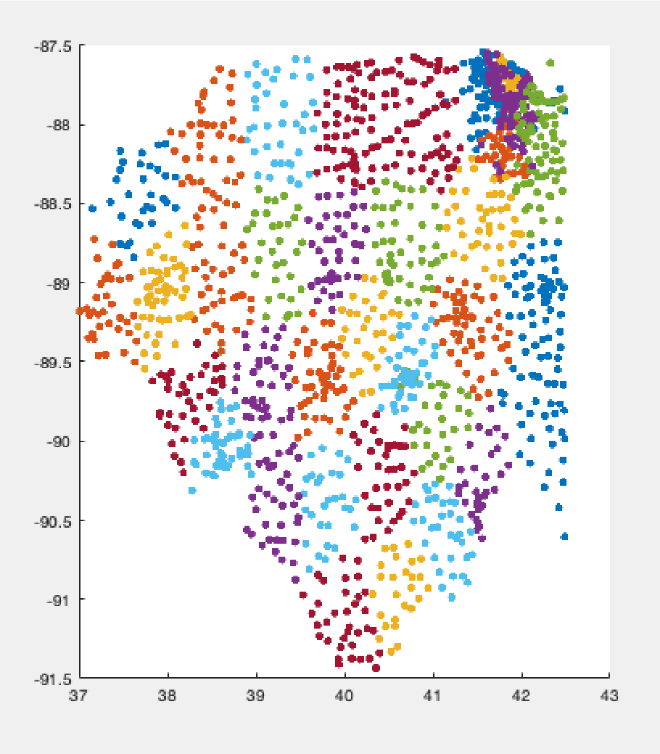
k- means with k = 20

Varying *k*

$$d(\text{Zipcode A, Zipcode B})^2 = (\text{Latitude A} - \text{Latitude B})^2 + (\text{Longitude A} - \text{Longitude B})^2 + (\text{Pop Density A} - \text{Pop Density B})^2/\text{stdev}(\text{Pop Density})^2$$



k=20



k=40

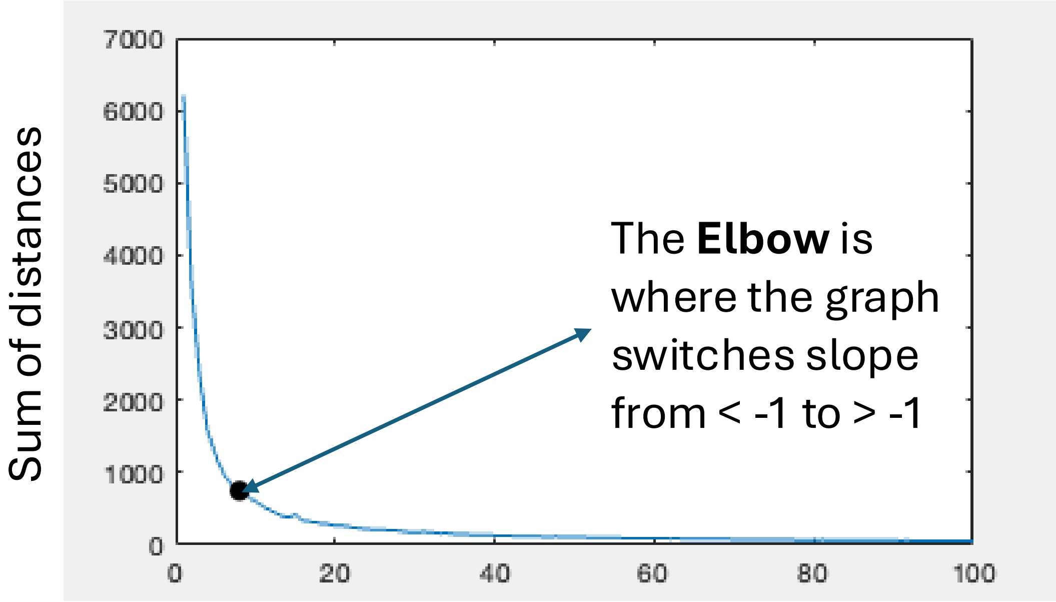
Choosing *k*

- *k* is a **tuning parameter**. Just like λ in Lasso:
  - *k* is a choice that needs to fit the application

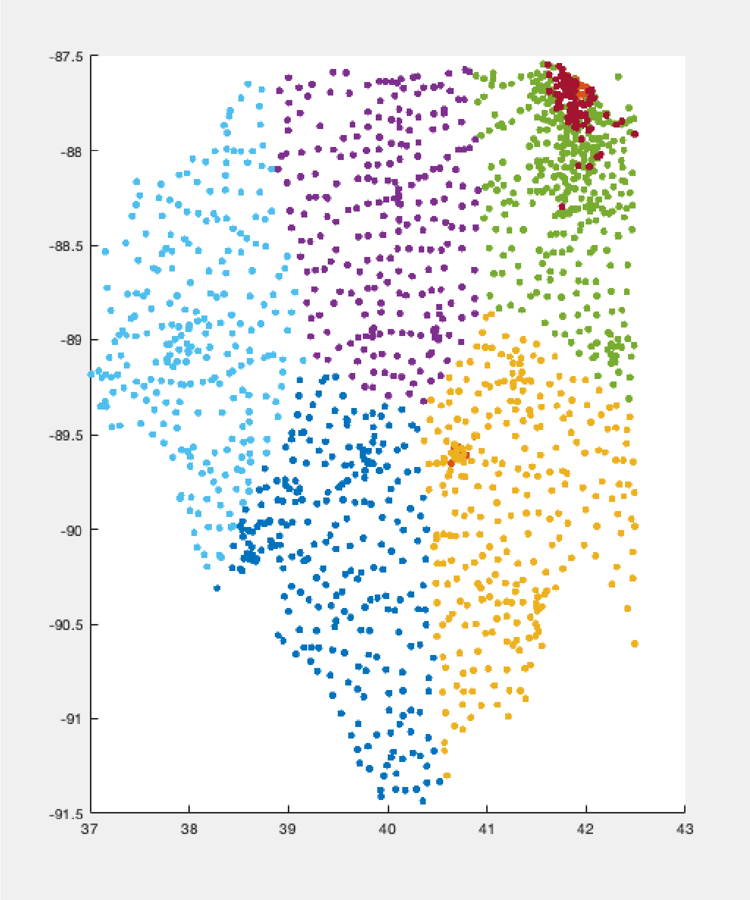
- One common rule for *k* is the Elbow rule
  - To implement, try different values of *k* (= 1,2,3,…), we where the objective function has an ”elbow” (see figure on following slides)
  - **Pros:** Simple to implement visually, if the data is very clearly defined clusters, these will be identified with the elbow rule
  - **Cons:** The Elbow rule is not application specific, and is indifferent to the final goal of the analysis

Choosing *k*

Sum of distances vs *k*



k



k = 7 *k*-means partition

The elbow rule is a heuristic for choosing the number of clusters

If possible, it is usally better to choose the number of clusters based on final application consideration

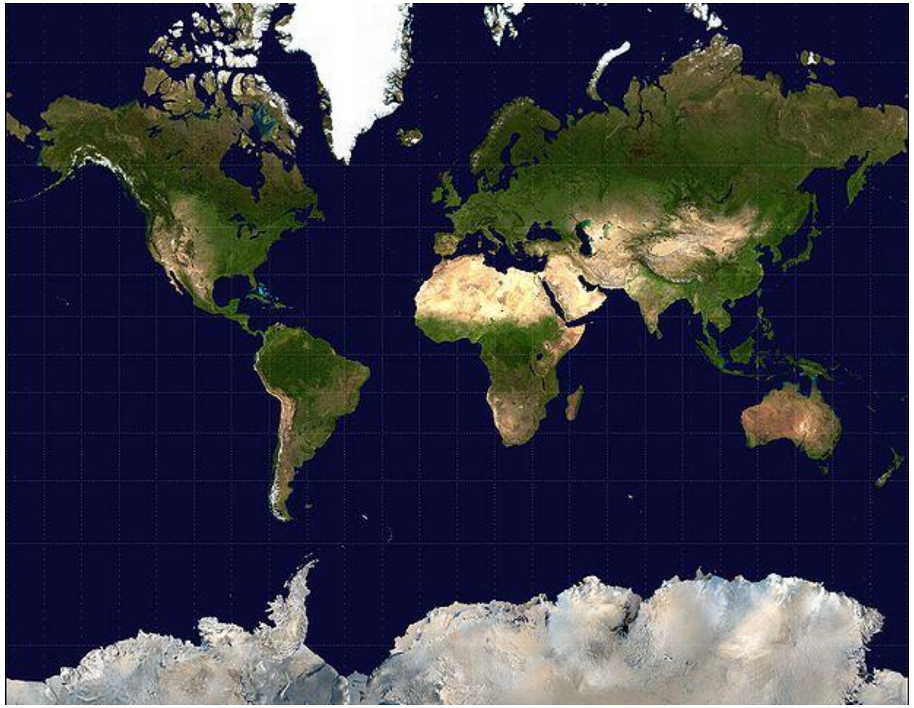
Note: specific application often require different cluster sizes than suggested by the elbow rule

Encoding

Part II – Encoding: Purpose, Methods, Applications

Part II: Encoding

- Encoding is any process of representing data in useful numerical ways (often but not always this means compressing the data)



Dimension reduction

- One common type of encoding is dimension reduction
- Data with 2 numerical attributes can be plotted in 2-D and visually inspected
- Data with more than 2 numerical attributes is hard to visualize
- Dimension reduction aims to find a 2-D (or lower D) representation

Linear dimension reduction

- A linear dimension reduction takes data attributes and returns a smaller number of attributes (e.g., 2) so that

$$(X_1, \dots , X_p) \rightarrow (Y_1, Y_2)$$

- The linear nomenclature means that each new attribute can be obtained as a linear combination (which means linear regression)

$$Y_1 = X_1\,\beta_1 + \dots + X_p\,\beta_p$$
$$Y_2 = X_1\,\gamma_1 + \dots + X_p\,\gamma_p$$

Principal components

- A method for calculating dimension reduction rules

- $(\beta_1, \ldots, \beta_p) = \text{argmax}\{\text{var}(Y_1)\}$

- Subject to: $\beta_1^2 + \ldots + \beta_p^2 = 1$

- $(\gamma_1, \ldots, \gamma_p) = \text{argmax}\{\text{var}(Y_2)\}$

- Subject to: $\gamma_1^2 + \ldots + \gamma_p^2 = 1$ and $\text{corr}(Y_1, Y_2) = 0$

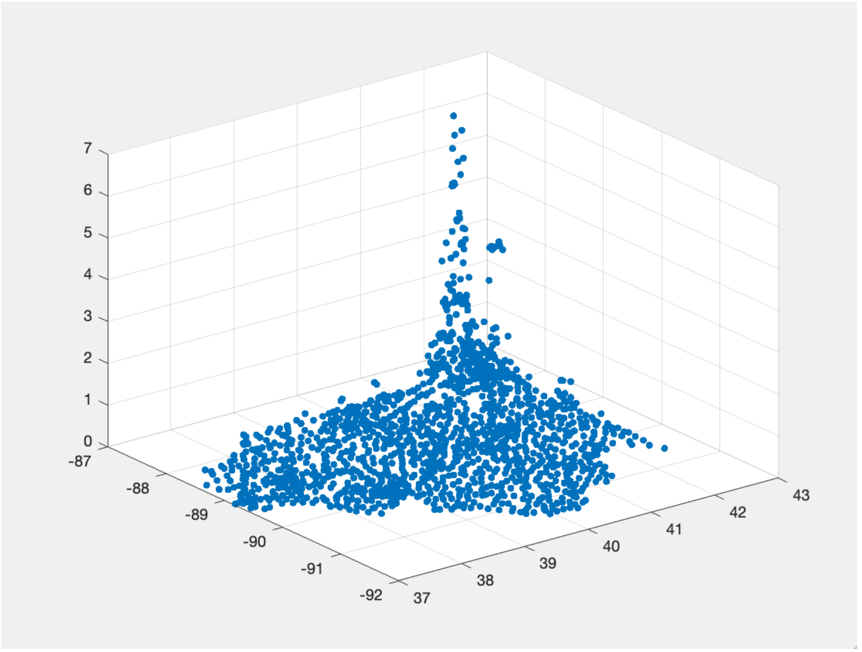
Generalizing to more components

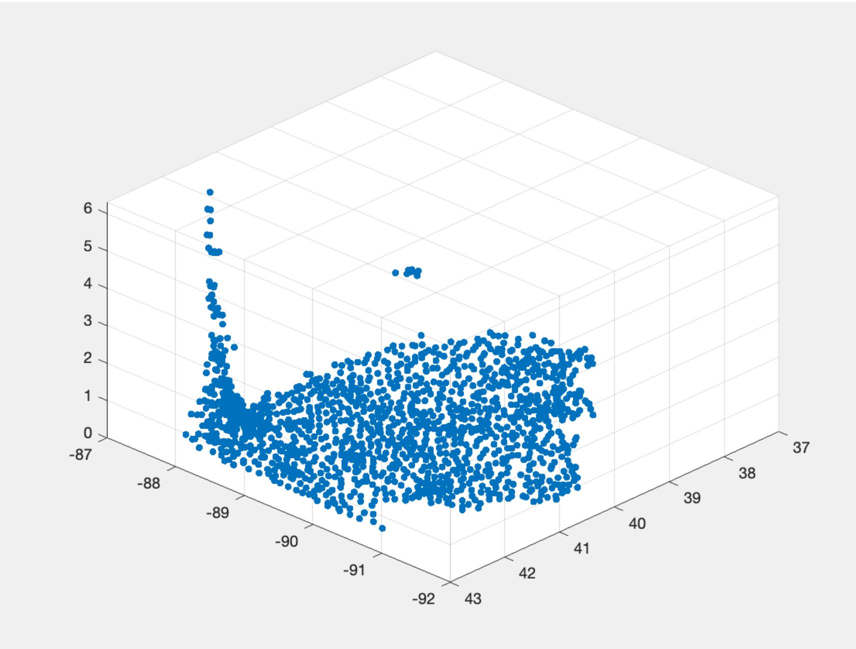
- Can generalize to 3 or 4 or more components
- Example: $(X_1, \ldots X_p) \rightarrow (Y_1, Y_2, Y_3, \ldots)$
  - Can still think of each dimension as a regression
    - $Y_1 = X_1 \beta_1 + \ldots + X_p \beta_p$
    - $Y_2 = X_1 \gamma_1 + \ldots + X_p \gamma_p$
    - $Y_3 = X_1 \delta_1 + \ldots + X_p \delta_p$
    - …
  - $(\delta_1, \ldots, \delta_p) = \text{argmax}\{\text{var}(Y_2)\}$
  - Subject to: $\delta_1^2 + \ldots + \delta_p^2 = 1$ and $\text{corr}(Y_3, Y_1) = 0$ and $\text{corr}(Y_3, Y_2) = 0$.
  - …
- And so forth (continuing up the Greek alphabet)
- The chosen number of components is again a tuning parameter

Applying Principal Components

- Illinois Zipcodes Data – Problem: Segment the zipcodes

  (x_1, x_2, x_3) = (Latitude, Longitude, Population Density)





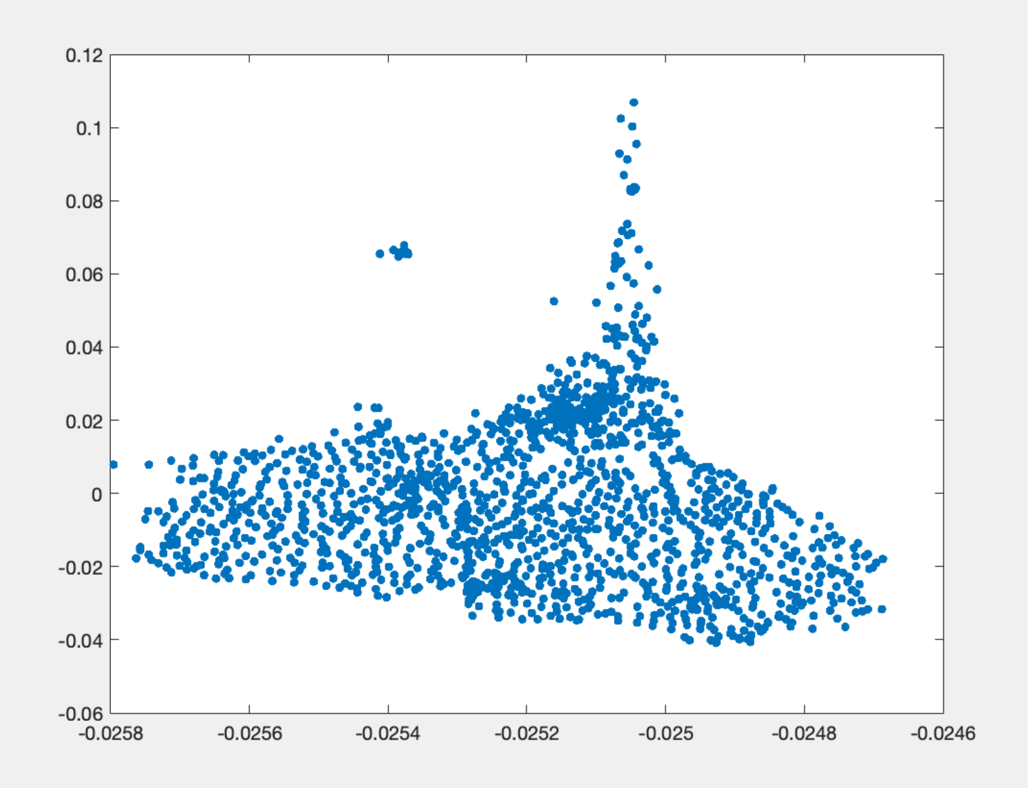
Different views – Principal components tries to choose the view that preserves the most information

Properties of principal components

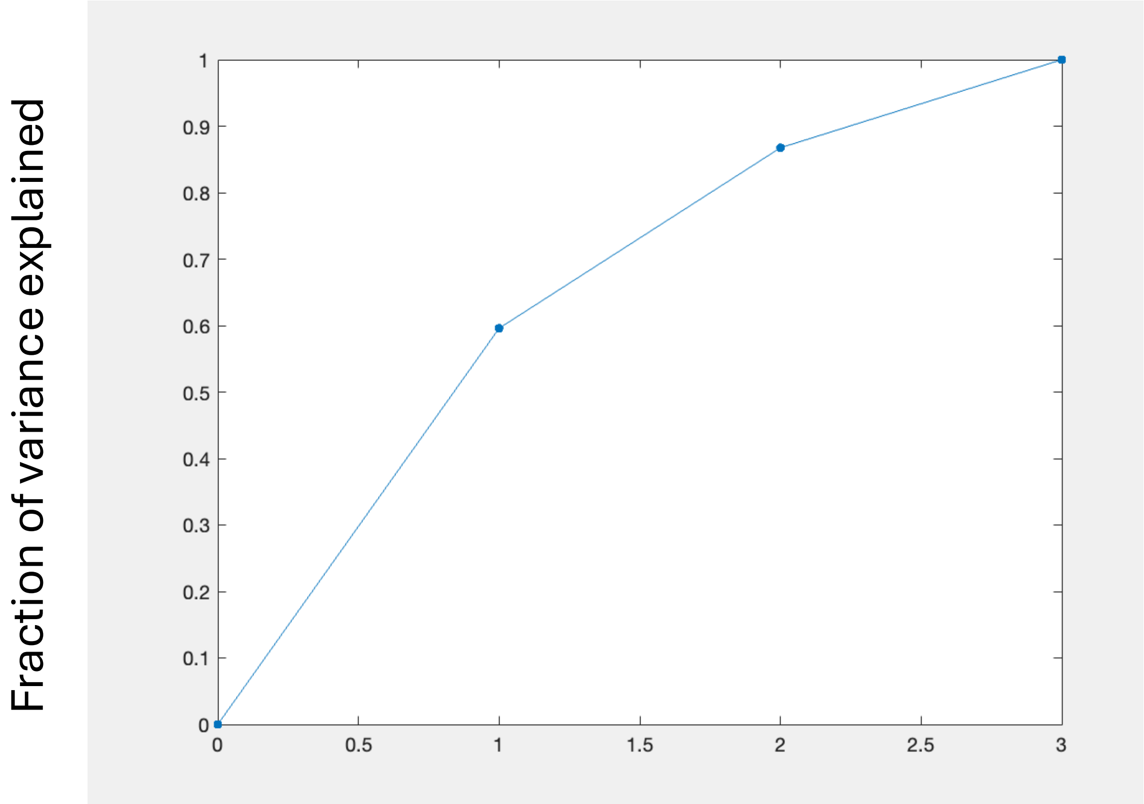
- Each principal compoment comes equipped with a variance σ_j^2
  - $\sigma_1{}^2 = \mathrm{var}\,(\,Y_1\,)$
  - $\sigma_2{}^2 = \mathrm{var}\,(\,Y_2\,)$
  - …
- σ_j^2 is interpreted as the fraction of variation explained by the j^{th} component
- $\sigma_1{}^2 + \sigma_2{}^2 + \ldots + \sigma_j^2$ is the fraction of variation explained by all the first j components. Principal components dimension reduction is useful when the fraction of variation explained is high (i.e., $F_j \approx 1$)

$$F_j = \text{Fraction of explained variance} = \frac{\sigma_1{}^2 + \sigma_2{}^2 + \ldots + \sigma_j^2}{\sigma_1{}^2 + \sigma_2{}^2 + \ldots + \sigma_j^2 + \sigma_{j+1}{}^2 + \ldots + \sigma_p{}^2}$$

First two principal components



Data Dimension-reduced to Two Principal Components



Number of Principal Components

Why principal components before *k*-means?

- Distances are not well behaved in high dimensions
- This problem is related to overfitting with sufficiently high degree polynomials in supervised learning
- By descending first to a lower dimension and then performing *k*-means, noise can be filtered out of the distance measure
- One can also descend down to 2-dimensions with the first 2 principal components and do clustering with the eyeball method (i.e., by visual inspection)

When to do dimension reduction before clustering?

- With many attributes and Euclidean distance,

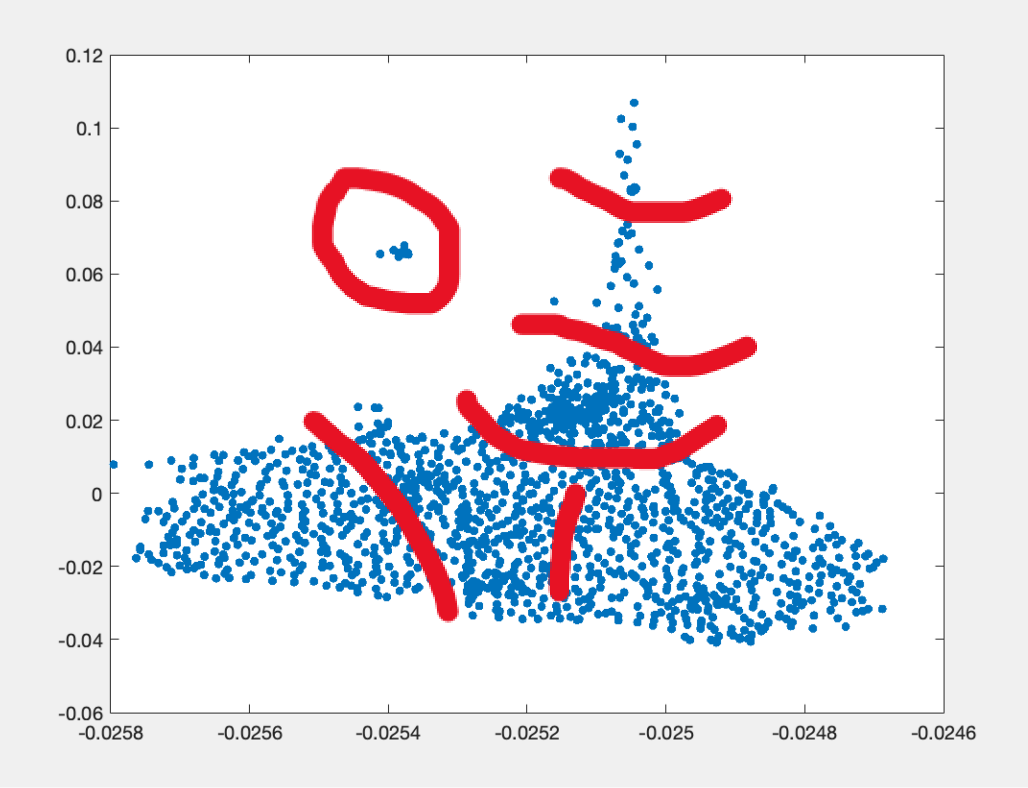
All observations are far away from each other

- The above is a general statement about Euclidean space – its implication is that *k* means is unstable without first doing dimension reduction
- This problem is related to overfitting with sufficiently high degree polynomials in supervised learning
- By descending first to a lower dimension and then performing *k*- means, noise can be filtered out of the distance measure
- One can also descend down to 2-dimensions with the first 2 principal components and do clustering with by visual inspection
- Note: there is a tradeoff- principal components does discard information. If data is not too high dimensional (3 or 4), *k*-means may still give reasonable results. In this case, doing *k*-means first then principal components for visualization may be more sensible

Clustering after principal components

Clustering method to the right is by visual inspection

k-means with 7 clusters yields approximately the same partition



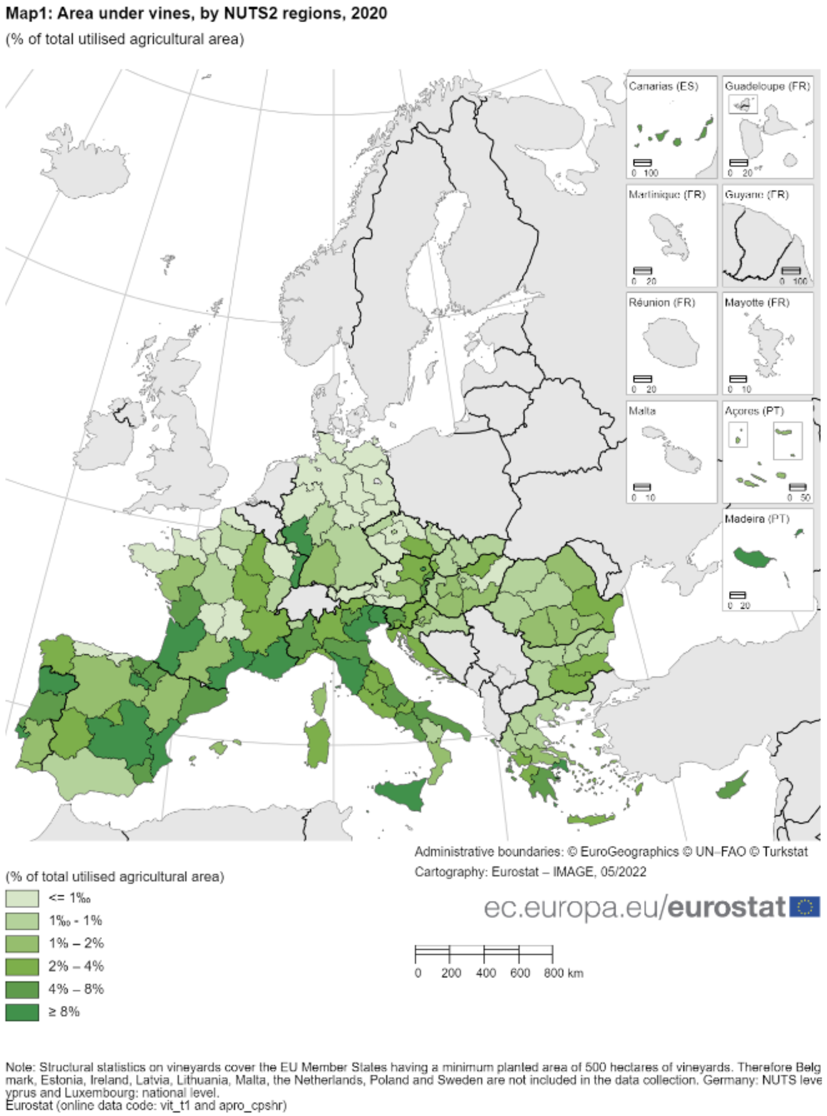
Nonlinear dimension reduction

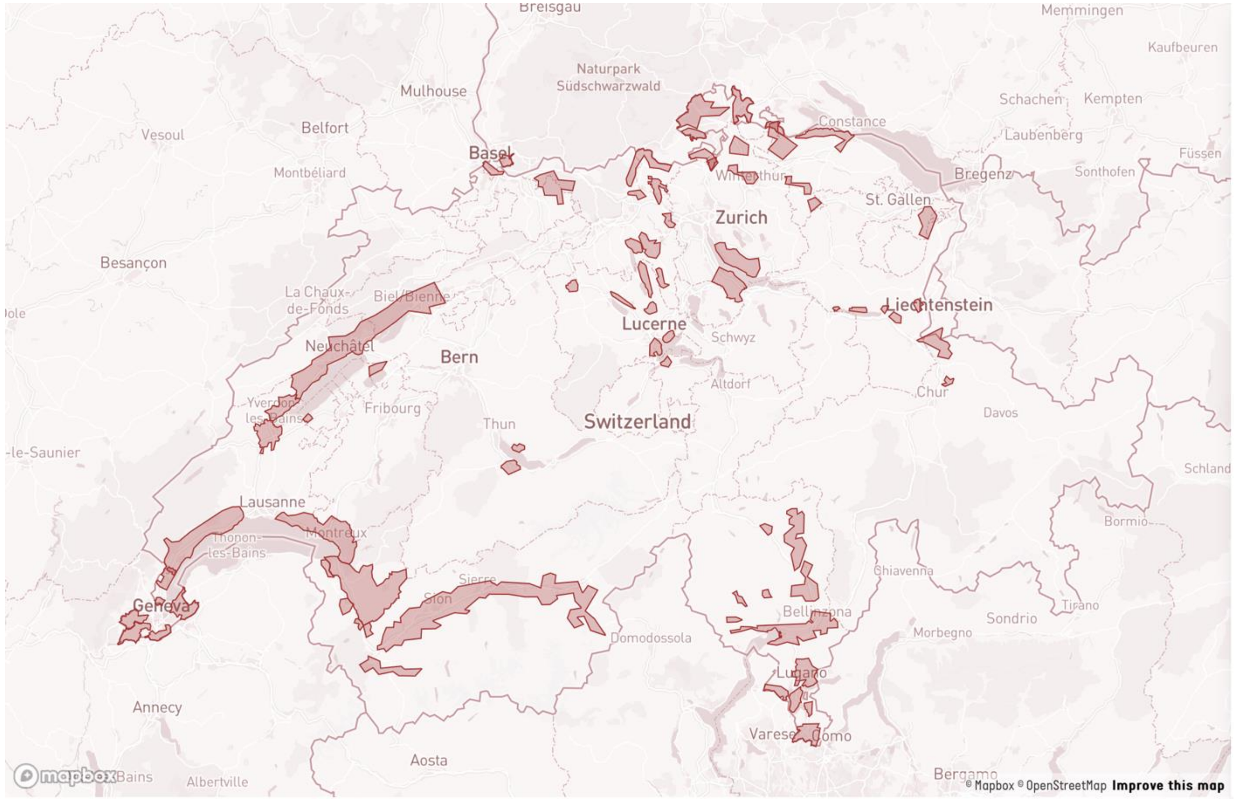
- Still assign $(X_1, \ldots X_p) \rightarrow (Y_1, Y_2, Y_3, \ldots)$
- $$\begin{aligned} Y_1 &= f_1(X_1,\ldots,X_p) \\ Y_2 &= f_2(X_1,\ldots,X_p) \\ &\ldots \\ Y_m &= f_m(X_1,\ldots,X_p) \end{aligned}$$
- The only difference is now, $f_1,\ldots,f_m$ do not need to be linear
- These include autoencoders and will be considered in upcoming lectures

Recommenders

Part III – Recommenders: Applications to wine recommendation systems, the Netflix problem, and the coaches problem

Application 2: Recommender systems for Swisswine





Questions

- **Question 1:** If I like the 2021 Räuschling from Schwarzenbach, what else might you recommend me?

- **Question 2:** Switzerland does not belong to the EU, and has laws which make export to other countries difficult (including the USA). If I like a particular EU wine, what Swiss wines might you recommend?

- **Question 3:** What other similarities are there between wines within clusters?

Example wine

Steckbrief

Erster Mémoire-Jahrgang
2005

Appellation
Zürichsee AOC

Rebsorte
Räuschling

Parzellen
0,75 ha in der Lage Seehalde in Meilen, Südexposition, 420 m ü. M.

Bodentyp
Sandsteinverwitterungsboden

Pflanzdichte
4200

Erziehungssystem
Guyot

Reben Pflanzjahr
Pflanzjahr der Reben 1989-2010

Kelterung
Entrappen, Pressen, statisch Entschleimen. Vergärung mit Reinzucht-hefen (1895) im Stahltank. Biologischer Säureabbau. 2 Filtrationen.

(Stand 2018)

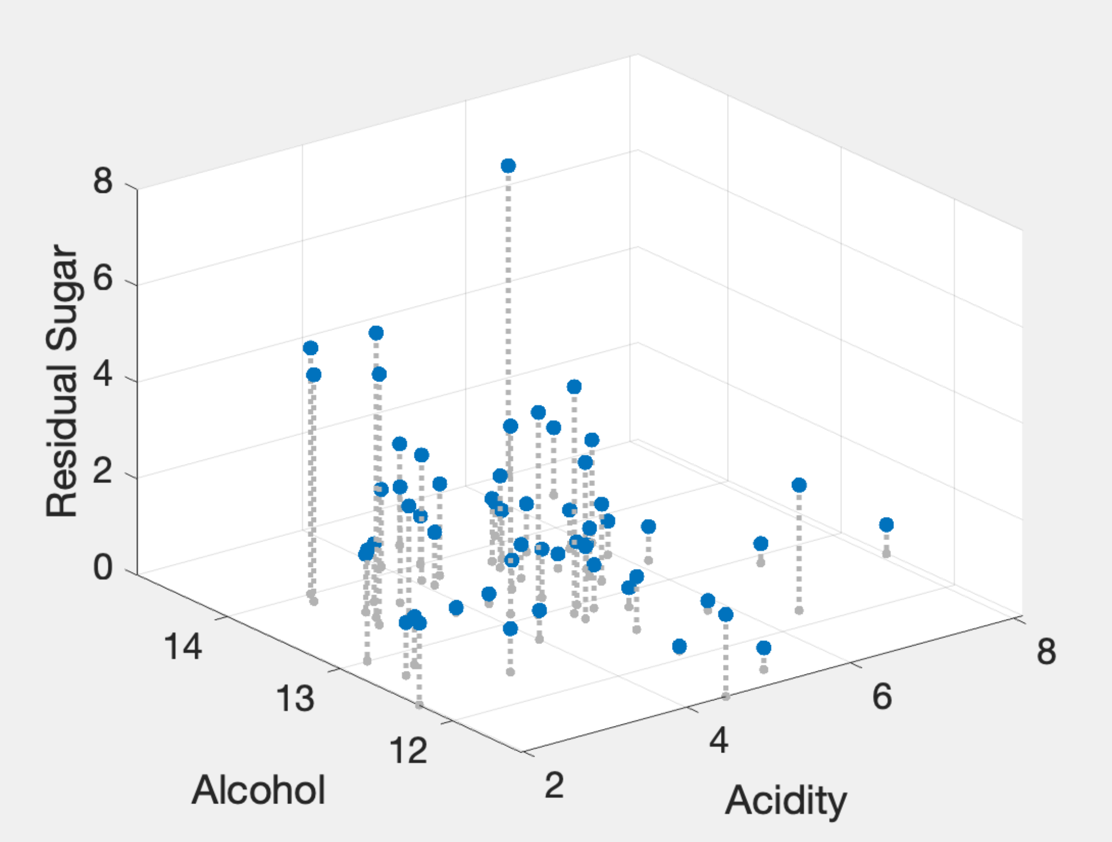
Räuschling:

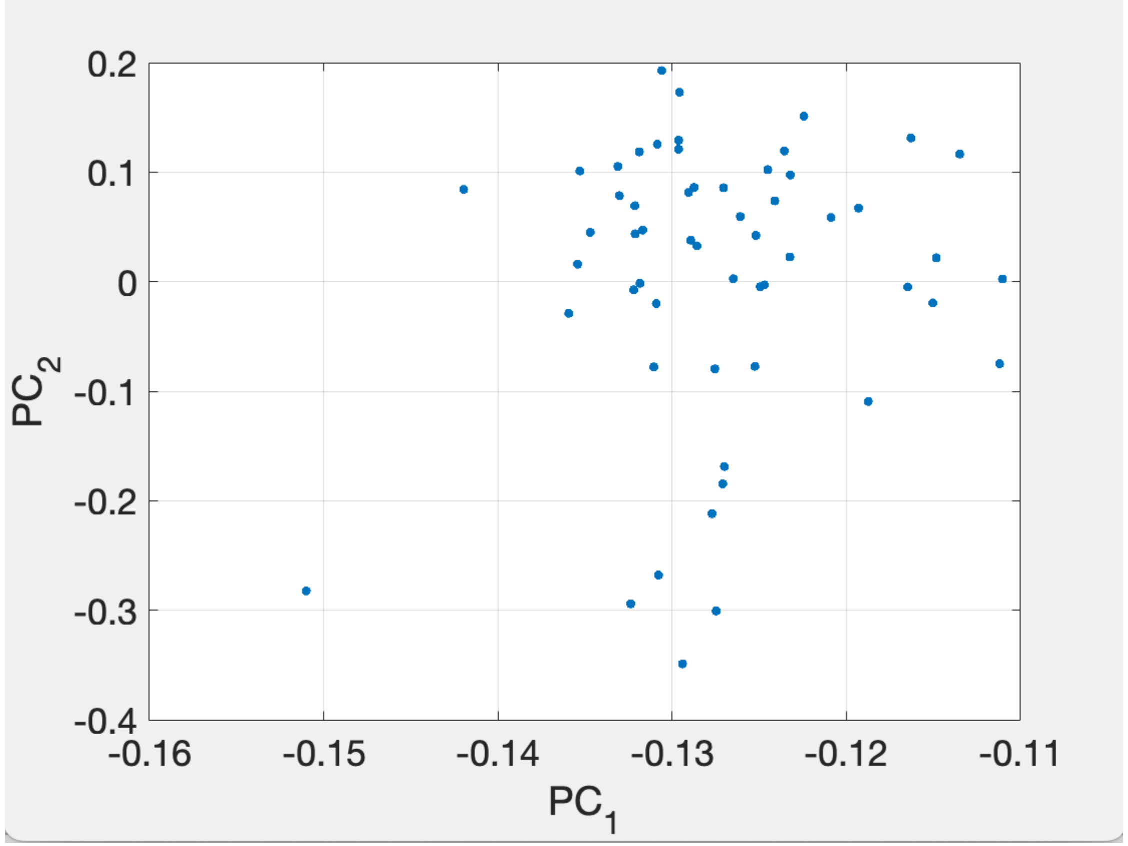
- A grape native to Zürich and grown exclusively grown there

Dataset variables (full dataset contains 60 wines and ~10 vintages per wine)

Weinbau	Alkohol	Gesatsäure	Restzucker	Rebsorte	Jahrgang	Region
Schwarzenbach	13.2	5.45	1.6	Räuschling	2022	Deutschweiz
St. Jodern Kellerei	13.6	3.54	2.4	Heida	2022	Wallis
Mounir Weine	13.4	3.37	2.3	Cave du Rhodan	2021	Wallis
Caves du Chambleau	13.8	4.94	1.3	Pinot Noir	2021	Drei-Seen Region
Grand'Cour	14	4.34	1	Cabernet Frac	2020	Genf
Pircher	14.1	5.39	0.7	Pinot Noir	2020	Deutschschweiz
Castello di Morcote	13.5	5.94	0.7	Merlot	2020	Tessin
Wegelin	13.3	6.16	0.7	Pinot Gris	2022	Deutschschweiz
Donatsch	14.8	6.52	6.4	Completer	2022	Deutschschweiz
Schmidheiny	13.5	5.33	0.3	Pinot Noir	2022	Deutschschweiz
Osterfingen	13	4.87	1.3	Pinot Blanc	2022	Deutschschweiz
Domaine des Muses	13.4	3.01	5.2	Humagne Blanche	2022	Wallis
Domaine des Muses	13.1	4.2	3.9	Humagne Blanche	2018	Wallis
Domaine des Muses	13	4.4	4.2	Humagne Blanche	2015	Wallis
Domaine des Muses	12.9	4.7	4.7	Humagne Blanche	2012	Wallis
Domaine du Clos des Pins	13.4	4.63	0.6	Gamaret	2021	Genf
Domaine du Clos des Pins	14.3	4.5	2.1	Gamaret	2018	Genf
Domaine du Clos des Pins	13.7	4.1	1.1	Gamaret	2015	Genf

Plotting the data – subsample containing most recent vintage from each wine





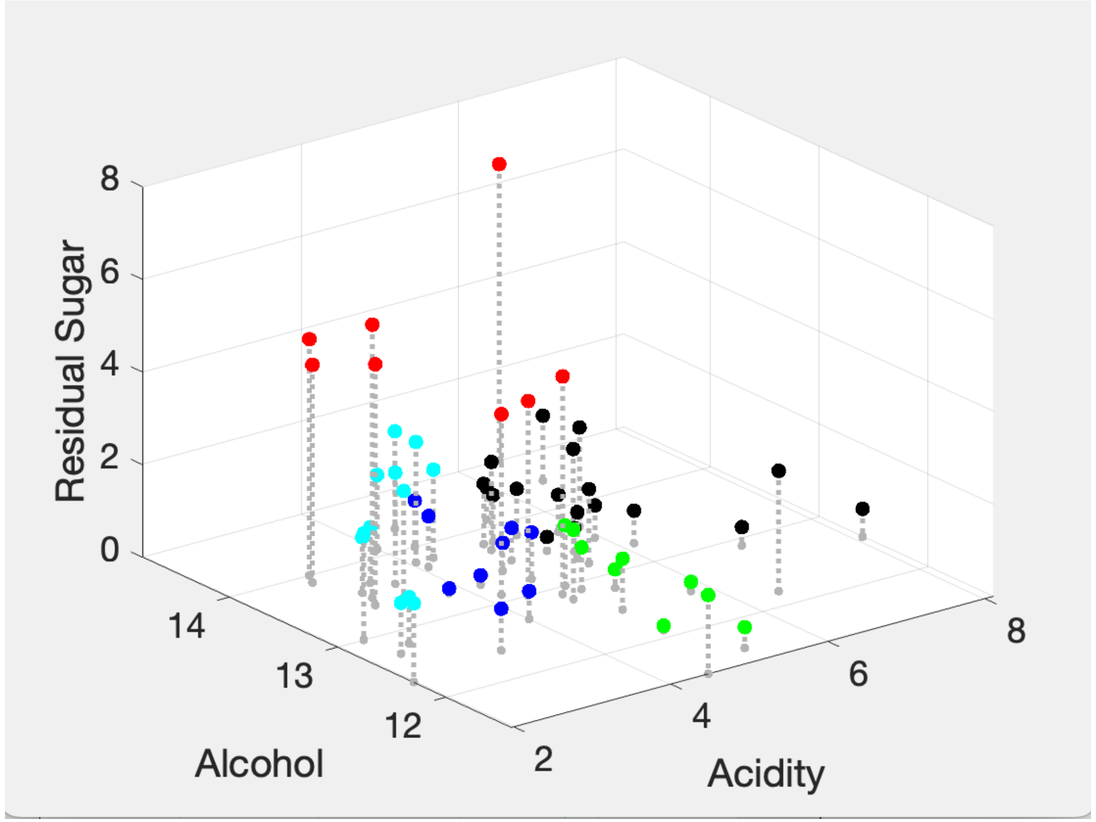
Full dataset together with first two principal components

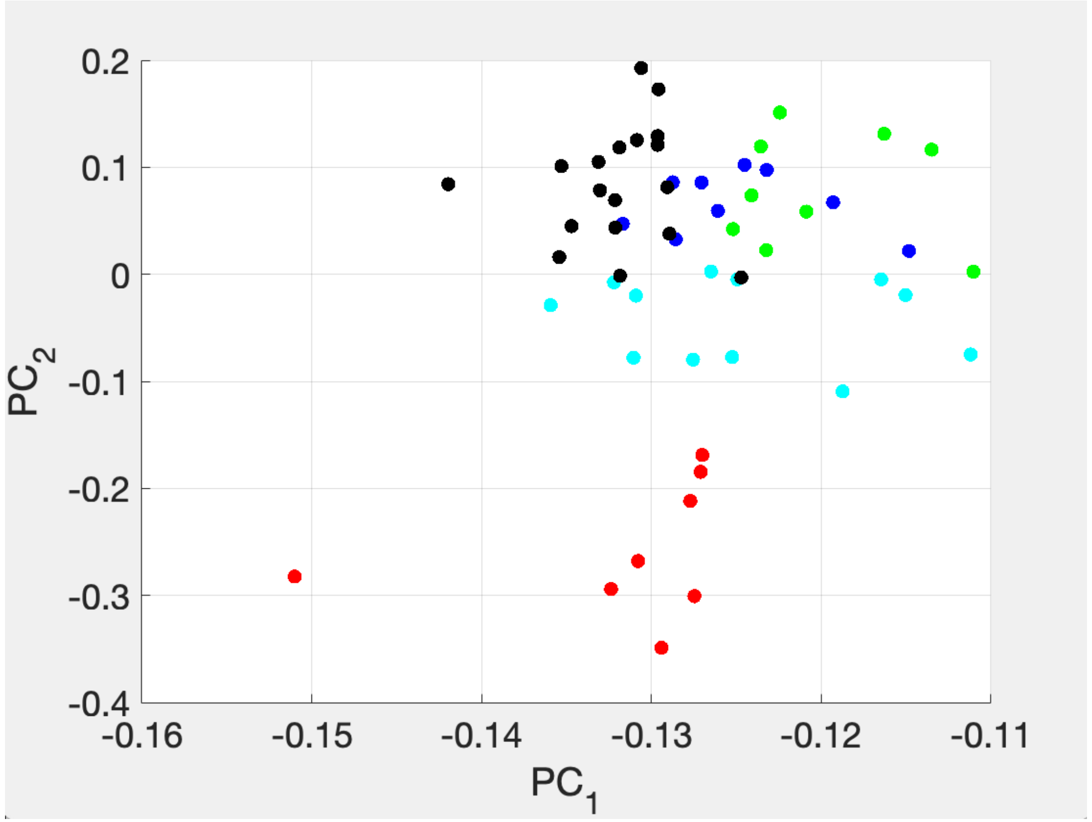
Defining a distance

- **Step 1**: List relevant observable numerical attributes are:
  - alc: Alcohol by volume (Alkohol)
  - acid: Acidity (Gesatzsäure)
  - sugar: Restzucker (Residual sugar)
- **Step 2:** Normalizing is possible, but visual inspection shows that all variables are on a similar scale
- **Step 3:** Define Euclidean distance.

$$d(i, j) = \sqrt{(\text{acid}(i) - \text{acid}(j))^2 + (\text{alc}(i) - \text{alc}(j))^2 + \cdots + (\text{sugar}(i) - \text{sugar}(j))^2}$$

Visualizing resulting clusters





- Cluster assignments for $k = 5$

Recommendations

- Suppose that a customer rates the 2021 Pinot Noir from 'Weingut Kastanienbaum AG' highly. Recommending based on cluster:

```
Recommendations using k=10
              Winery                      Rebsorte        Jahrgang          Region
    ____________________________     ______________    ________    __________________

    {'Schmidheiny'              }    {'Pinot Noir'}      2022      {'Deutschschweiz'}
    {'Domaine du Clos des Pins'}     {'Gamaret'    }     2021      {'Genf'           }
    {'Schlossgut Bachtobel'     }    {'Pinot Noir'}      2021      {'Deutschschweiz'}
    {'Weingut Wolfer'           }    {'Pinot Noir'}      2021      {'Deutschschweiz'}

Recommendations using k=20
            Winery                  Rebsorte        Jahrgang          Region
    ________________________    ______________    ________    __________________

    {'Castello di Morcote' }    {'Merlot'     }     2020      {'Tessin'         }
    {'Wegelin'             }    {'Pinot Gris'}      2022      {'Deutschschweiz'}
    {'Schlossgut Bachtobel'}    {'Pinot Noir'}      2021      {'Deutschschweiz'}
```

*Note: using k=5 yields 30 wines – perhaps this is too many? Choose between k=10 and k=20

Problem: The Netflix Problem

- Given a partial dataset of ratings, populate the entire dataset in the best way

	Terminator II	2001: A Space Odyssey	Bambi	Gladiator
Customer 1	★★★★	★★		★★★★★
Customer 2	★★★		★★★★★	
Customer 3	★	★★★★★		
Customer 4	★★★★	★★	★★★★	★★★★
Customer 5		★★	★★★★★	★★★★

Challenge: Populate the blanks

- How to know if a given set of guessed ratings (★s) is the best possible?

	Terminator II	2001: A Space Odyssey	Bambi	Gladiator
Customer 1	★★★★	★★	★★★★	★★★★★
Customer 2	★★★	★★★★★	★★★★★	★★★★
Customer 3	★	★★★★★	★★	★★
Customer 4	★★★★	★★	★★★★	★★★★
Customer 5	★★★★	★★	★★★★★	★★★★

One solution using principal components

- Let X be all observed values (i.e., the entire dataset – the gold colored ★s)
- Let Y be all estimated values (i.e., the entire collection of estimated ratings– the grey colored ★s)
- The new dataset Y has a fraction of explained variation from its first 3 principal components

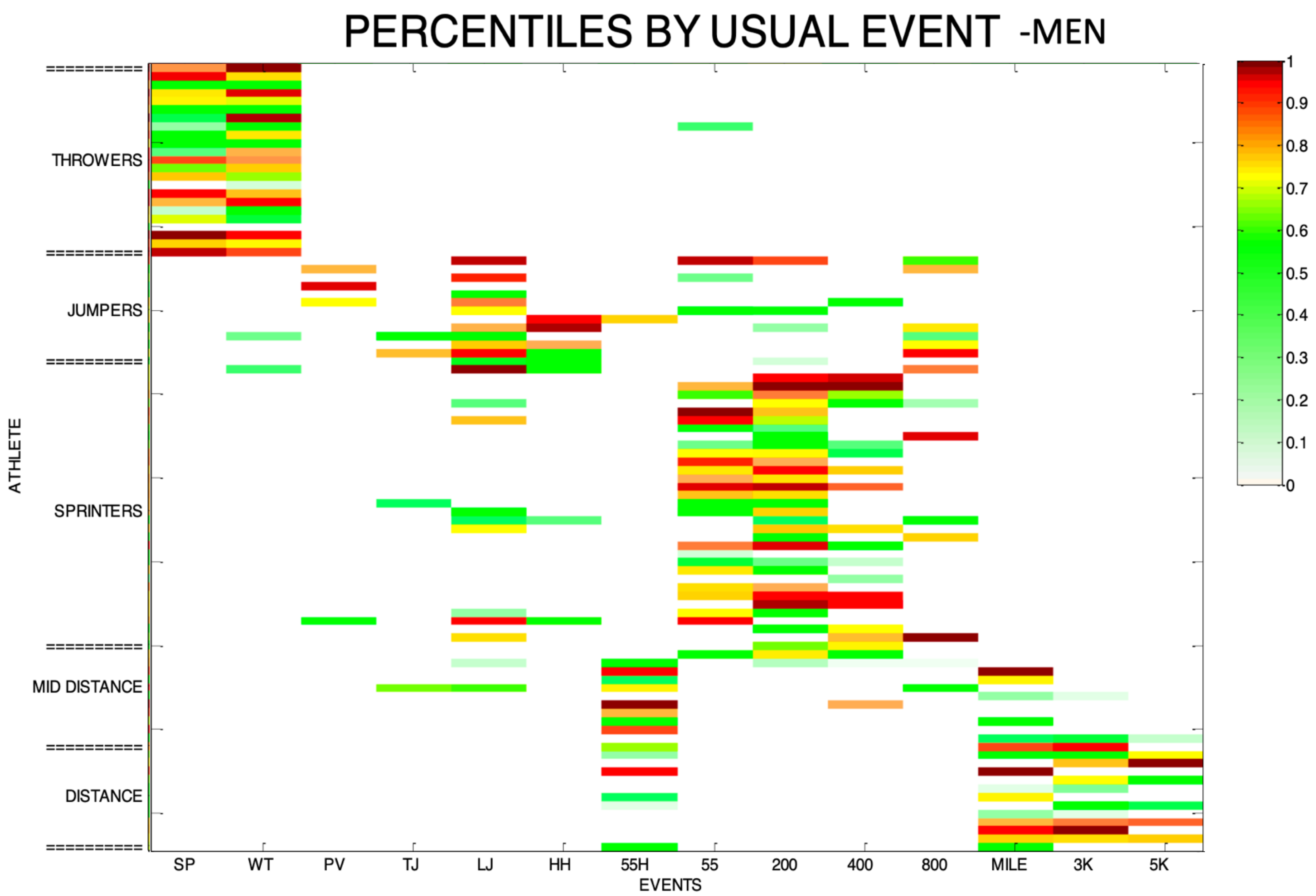
$$Y = \text{argmax}\, L(U), \quad \text{with } L(U) = F_3$$

Such that $X_{ij} = U_{ij}$ whenever customer i rated movie j.

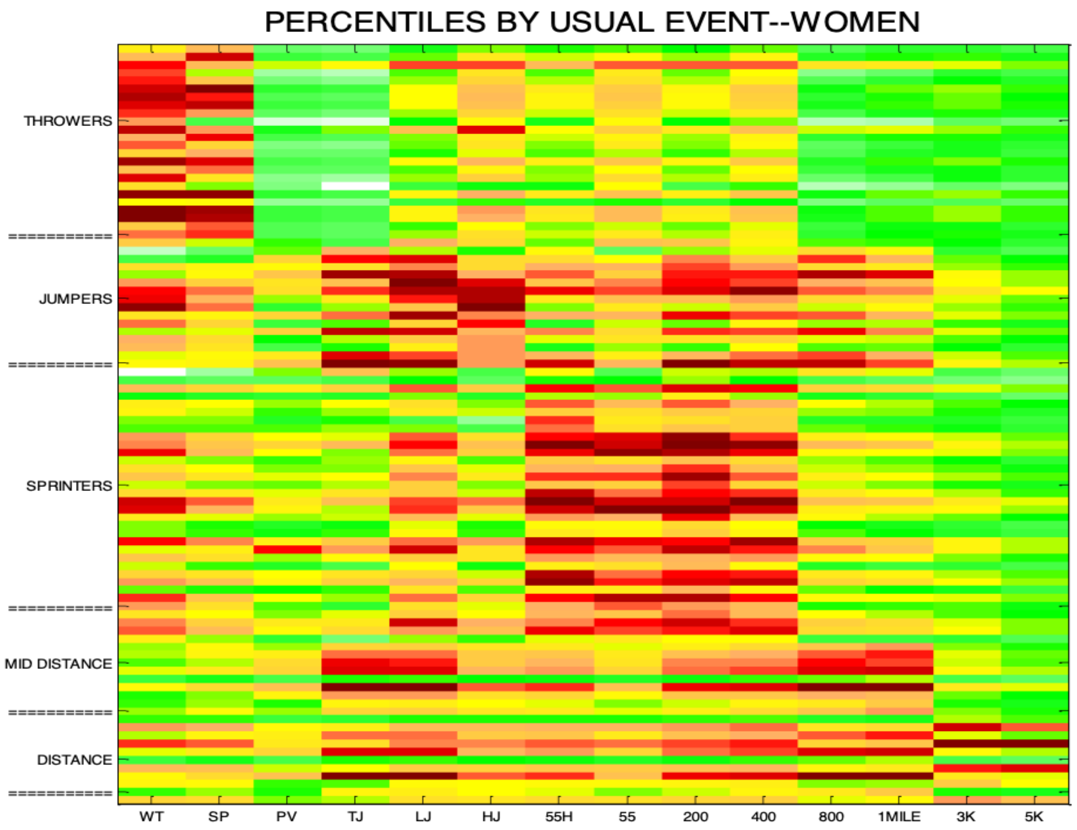
- The constraint means we search for predicted ratings that match datasets
- Recall F_3 is the fraction of variance explained by first 3 principal components
- The choice of 3 is a tuning parameter. Other small numbers can be used.

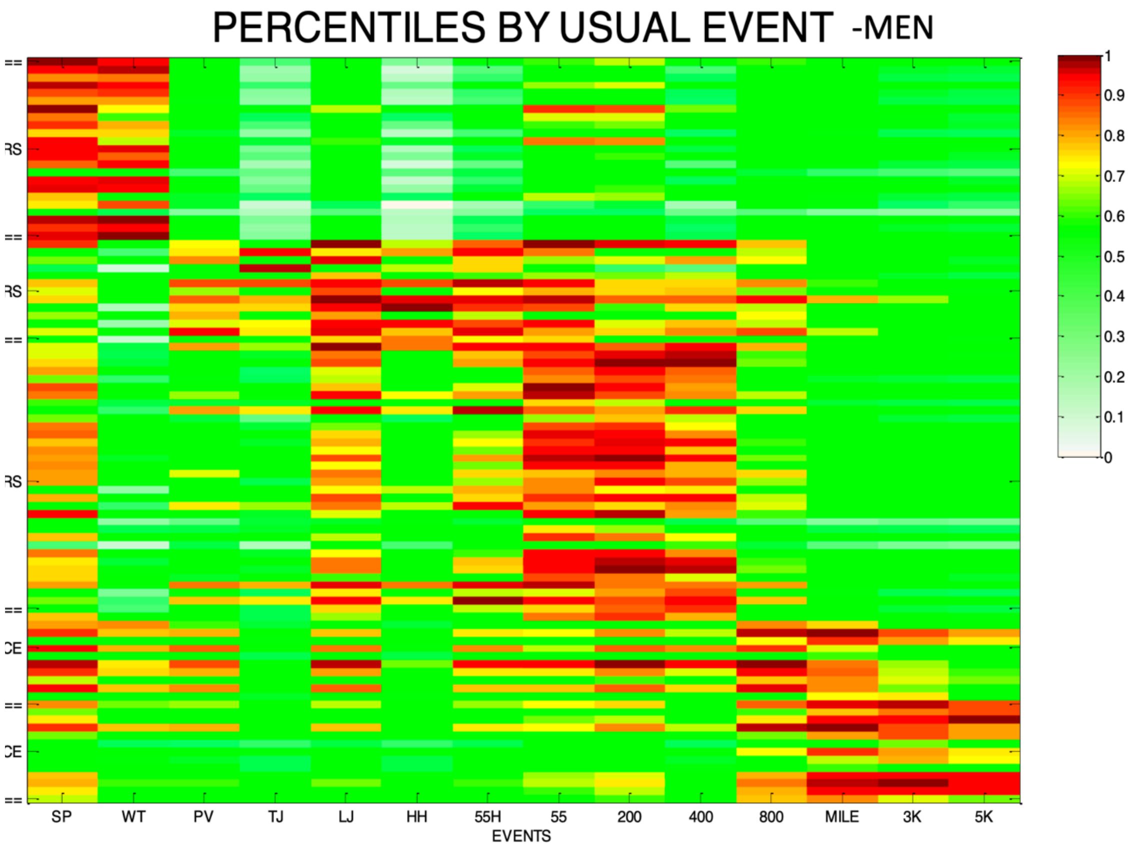
Application 3: performance data track and field

- Data from UChicago Athletics website
- All indoor home meets 2008
- Athletes from 31 different schools
- Athletes competed ≥ 4 times: total 93 male and 301 female.
- Weight throw, shot put, pole vault, triple jump, long jump, high jump, 55m hurdles, 55m, 200m, 400m, 800m, 1 mile, 3km, 5km.



The track coaches problem: build the right team with partial observations of event performance





Cross Validation	Mean Absolute Error for Percentile	Mean Relative Error for Raw Performance
Men	15.96%	4.44%
Women	10.96%	7.08%

Additional References

- Textbook reference

  G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor (2023). *An Introduction to Statistical Learning withApplications in Python* [Chapter 12].

  Available for download at https://www.statlearning.com

- Reading about the history of the Netflix problem

  R. Bell and Y. Korin (2007). *Lessons from the Netflix Problem* (by Robert Bell and Yehuda Korin)

  Available for download at https://dl.acm.org/doi/pdf/10.1145/1345448.1345465